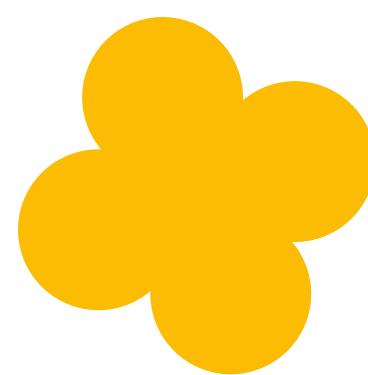




Hands-on:

Transform your agent

Let's take the default agent from module 1 and transform it into a specialized math tutor.

Starting point (module 1 default)

```
Python
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description='A helpful assistant agent.',
    instruction='You are a helpful assistant.'
)
```



Step 1: Update the internal name

Make the name more descriptive for what this agent actually does:

```
Python
root_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent', # More specific internal name
    description='A helpful assistant agent.',
    instruction='You are a helpful assistant.'
)
```



Step 2: Write a specific description

Remember: description tells other agents what this agent does (important for multi-agent systems).

Python

```
root_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent',
    description='Helps students learn algebra by guiding them through problem-
solving steps.',
    instruction='You are a helpful assistant.'
)
```

What changed: Other agents can now understand this is specifically for algebra tutoring, not general questions.

Step 3: Write a simple instruction

Remember: `instruction` tells this agent how to behave and respond.

Python

```
root_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent',
    description='Helps students learn algebra by guiding them through problem-
solving steps.',
    instruction='You are a patient math tutor. Help students with algebra problems.'
)
```

What this instruction does:

- Defines the agent's role (math tutor)
- Sets the personality (patient)
- Specifies the task (help with algebra)

Note: This simple instruction is perfect for learning the basics. In course 3, you'll learn how to write production-ready instructions with detailed persona, boundaries, and examples.

Complete transformed agent

Python

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent',
    description='Helps students learn algebra by guiding them through problem-  
solving steps.',
    instruction='You are a patient math tutor. Help students with algebra problems.'
)
```

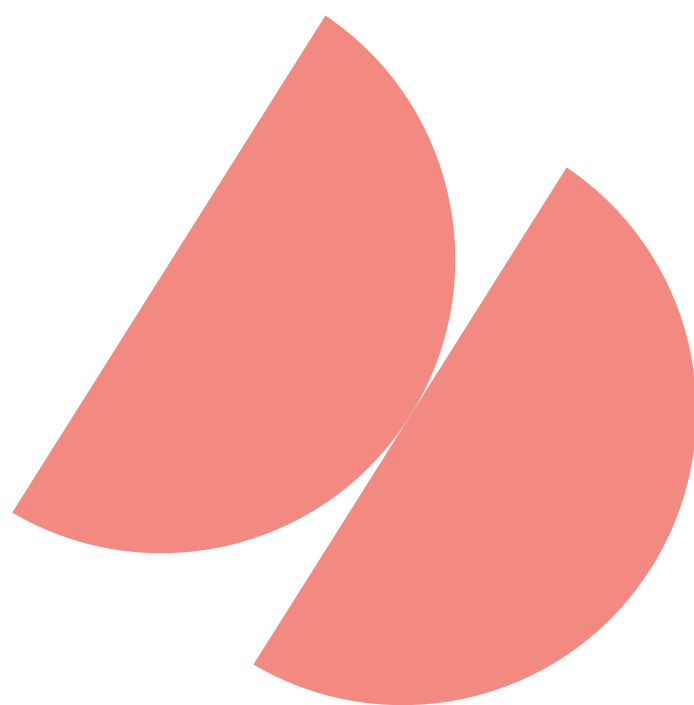
Save this to your `agent.py` file in your `my_first_agent` directory from module 1.

This simple agent demonstrates:

- ✓ Specific role definition (math tutor)
- ✓ Clear personality (patient)
- ✓ Defined task scope (algebra problems)

Are you ready for more?

Course 3 will teach you how to enhance this with professional instruction patterns, boundaries, and examples for production use.



Test your customized agent

Reference: [ADK docs – Python Quickstart: Interacting with Your Agent](#)

Now, let's see your customized math tutor agent in action using the ADK web interface.

Step 1: Start the web interface

From your `my_first_agent` directory (or the parent directory containing it), run:

```
Shell  
adk web
```

Expected output:

```
None  
INFO:      Started server process  
INFO:      Waiting for application startup.  
INFO:      Application startup complete.  
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

What this does:

- Starts a local development server at <http://localhost:8000>
- Automatically opens your browser to the ADK web interface
- Loads your customized `math_tutor_agent` from `agent.py`

From ADK docs:

“You can start the web interface using the following command: `adk web`. This command starts a web server with a chat interface for your agent.”



Step 2: Interact with your math tutor

Once the web interface opens:

1. Select your agent from the dropdown (it should show your agent's [name](#))
2. Try the following test questions to see how your customized instructions work:

Example interactions:

Question: "What is $2x + 5 = 13$?"

Expected behavior: Your agent should:

- Act as a math tutor
- Help with the problem
- Demonstrate a patient tone

Question: "I don't understand algebra at all."

Expected behavior: Your agent should:

- Respond as a patient tutor
- Offer help with algebra concepts
- Maintain a supportive approach

Step 3: Observe the difference

Compare your customized agent's responses to the generic "You are a helpful assistant" instruction from module 1.

Generic agent (module 1):

```
Python
instruction='You are a helpful
assistant.'
```

- Generic responses
- No specific domain expertise

Customized math tutor (module 2):

```
Python
instruction='You are a patient math
tutor. Help students with algebra
problems.'
```

- Focused on math tutoring
- Patient approach to teaching
- Specialized for algebra help

This demonstrates how even simple, focused instructions can significantly improve agent behavior.

Step 4: Stop the server

When you're done testing:

- Press [Ctrl+C](#) in the terminal to stop the web server

Troubleshooting:

If the agent doesn't appear in the dropdown:

- Make sure you're running [adk web](#) from the parent directory of [my_first_agent](#)
- Or run `adk web my_first_agent` to specify the path

If you see the old generic responses:

- Make sure you saved the changes to [agent.py](#)
- Stop the server ([Ctrl+C](#)), and restart [adk web](#)
- The server needs to be restarted to pick up code changes

Key takeaways

Required parameters:

- Only `model` and `name` are technically required
- But `instruction` is critical for useful behavior
- `description` is important for multi-agent systems

Best practices for course 2:

- Write simple, focused instructions that define the role and task
- Be specific about what the agent does (don't just "be helpful")
- Include a personality trait to guide the tone
- Test with `adk web` to see how instructions affect behavior

Common pitfalls:

- ✗ Vague instructions, like "be helpful"
- ✗ Confusing `description` with `instruction`
- ✗ Forgetting to assign to `root_agent` variable
- ✗ Using unclear or generic names

Next level: For production-ready agents, course 3 teaches advanced instruction patterns, including multi-section structures, boundaries, and few-shot examples.

